

An Introduction to JavaScript

Let's see what's so special about JavaScript, what we can achieve with it, and what other technologies play well with it.

What is JavaScript?

JavaScript was initially created to “make web pages alive”.

The programs in this language are called *scripts*. They can be written right in a web page's HTML and run automatically as the page loads.

Scripts are provided and executed as plain text. They don't need special preparation or compilation to run.

In this aspect, JavaScript is very different from another language called [Java](#).

Why is it called JavaScript?

When JavaScript was created, it initially had another name: “LiveScript”. But Java was very popular at that time, so it was decided that positioning a new language as a “younger brother” of Java would help.

But as it evolved, JavaScript became a fully independent language with its own specification called [ECMAScript](#), and now it has no relation to Java at all.

Today, JavaScript can execute not only in the browser, but also on the server, or actually on any device that has a special program called [the JavaScript engine](#).

The browser has an embedded engine sometimes called a “JavaScript virtual machine”.

Different engines have different “codenames”. For example:

- [V8](#) – in Chrome, Opera and Edge.
- [SpiderMonkey](#) – in Firefox.
- ...There are other codenames like “Chakra” for IE, “JavaScriptCore”, “Nitro” and “SquirrelFish” for Safari, etc.

The terms above are good to remember because they are used in developer articles on the internet. We'll use them too. For instance, if “a feature X is supported by V8”, then it probably works in Chrome, Opera and Edge.

How do engines work?

Engines are complicated. But the basics are easy.

1. The engine (embedded if it's a browser) reads (“parses”) the script.
2. Then it converts (“compiles”) the script to machine code.
3. And then the machine code runs, pretty fast.

The engine applies optimizations at each step of the process. It even watches the compiled script as it runs, analyzes the data that flows through it, and further optimizes the machine code based on that knowledge.

What can in-browser JavaScript do?

Modern JavaScript is a “safe” programming language. It does not provide low-level access to memory or the CPU, because it was initially created for browsers which do not require it.

JavaScript’s capabilities greatly depend on the environment it’s running in. For instance, [Node.js](#) supports functions that allow JavaScript to read/write arbitrary files, perform network requests, etc.

In-browser JavaScript can do everything related to webpage manipulation, interaction with the user, and the webserver.

For instance, in-browser JavaScript is able to:

- Add new HTML to the page, change the existing content, modify styles.
- React to user actions, run on mouse clicks, pointer movements, key presses.
- Send requests over the network to remote servers, download and upload files (so-called [AJAX](#) and [COMET](#) technologies).
- Get and set cookies, ask questions to the visitor, show messages.
- Remember the data on the client-side (“local storage”).

What CAN’T in-browser JavaScript do?

JavaScript’s abilities in the browser are limited to protect the user’s safety. The aim is to prevent an evil webpage from accessing private information or harming the user’s data.

Examples of such restrictions include:

- JavaScript on a webpage may not read/write arbitrary files on the hard disk, copy them or execute programs. It has no direct access to OS functions.

Modern browsers allow it to work with files, but the access is limited and only provided if the user does certain actions, like “dropping” a file into a browser window or selecting it via an `<input>` tag.

There are ways to interact with the camera/microphone and other devices, but they require a user’s explicit permission. So a JavaScript-enabled page may not sneakily enable a web-camera, observe the surroundings and send the information to the [NSA](#).

- Different tabs/windows generally do not know about each other. Sometimes they do, for example when one window uses JavaScript to open the other one. But even in this case, JavaScript from one page may not access the other page if they come from different sites (from a different domain, protocol or port).

This is called the “Same Origin Policy”. To work around that, *both pages* must agree for data exchange and must contain special JavaScript code that handles it. We’ll cover that in the tutorial.

This limitation is, again, for the user’s safety. A page from `http://anysite.com` which a user has opened must not be able to access another browser tab with the URL `http://gmail.com`, for example, and steal information from there.

- JavaScript can easily communicate over the net to the server where the current page came from. But its ability to receive data from other sites/domains is severely limited. Though possible, it requires explicit agreement (expressed in HTTP headers) from the remote side. Once again, that's a safety limitation.

Such limitations do not exist if JavaScript is used outside of the browser, for example on a server. Modern browsers also allow plugins/extensions which may ask for extended permissions.

What makes JavaScript unique?

There are at least *three* great things about JavaScript:

- Full integration with HTML/CSS.
- Simple things are done simply.
- Supported by all major browsers and enabled by default.

JavaScript is the only browser technology that combines these three things.

That's what makes JavaScript unique. That's why it's the most widespread tool for creating browser interfaces.

That said, JavaScript can be used to create servers, mobile applications, etc.

Languages “over” JavaScript

The syntax of JavaScript does not suit everyone's needs. Different people want different features.

That's to be expected, because projects and requirements are different for everyone.

So, recently a plethora of new languages appeared, which are *transpiled* (converted) to JavaScript before they run in the browser.

Modern tools make the transpilation very fast and transparent, actually allowing developers to code in another language and auto-converting it “under the hood”.

Examples of such languages:

- [CoffeeScript](#) is “syntactic sugar” for JavaScript. It introduces shorter syntax, allowing us to write clearer and more precise code. Usually, Ruby devs like it.
- [TypeScript](#) is concentrated on adding “strict data typing” to simplify the development and support of complex systems. It is developed by Microsoft.
- [Flow](#) also adds data typing, but in a different way. Developed by Facebook.
- [Dart](#) is a standalone language that has its own engine that runs in non-browser environments (like mobile apps), but also can be transpiled to JavaScript. Developed by Google.
- [Brython](#) is a Python transpiler to JavaScript that enables the writing of applications in pure Python without JavaScript.

- [Kotlin](#) is a modern, concise and safe programming language that can target the browser or Node.

There are more. Of course, even if we use one of these transpiled languages, we should also know JavaScript to really understand what we're doing.

Bubbling and capturing

Let's start with an example.

This handler is assigned to `<div>`, but also runs if you click any nested tag like `<em>` or `<code>`:

```
<div onclick="alert('The handler!')">
  <em>If you click on <code>EM</code>, the handler on <code>DIV</code> runs.</em>
</div>
```

Isn't it a bit strange? Why does the handler on `<div>` run if the actual click was on `<em>`?

Bubbling

The bubbling principle is simple.

When an event happens on an element, it first runs the handlers on it, then on its parent, then all the way up on other ancestors.

Let's say we have 3 nested elements `FORM > DIV > P` with a handler on each of them:

```
<style>
  body * {
    margin: 10px;
    border: 1px solid blue;
  }
</style>

<form onclick="alert('form')">FORM
  <div onclick="alert('div')">DIV
    <p onclick="alert('p')">P</p>
  </div>
</form>
```

A click on the inner `<p>` first runs onclick:

1. On that `<p>`.
2. Then on the outer `<div>`.
3. Then on the outer `<form>`.

4. And so on upwards till the document object.

So if we click on `<p>`, then we'll see 3 alerts: `p` → `div` → `form`.

The process is called “bubbling”, because events “bubble” from the inner element up through parents like a bubble in the water.

Almost all events bubble.

The key word in this phrase is “almost”.

For instance, a focus event does not bubble. There are other examples too, we'll meet them. But still it's an exception, rather than a rule, most events do bubble.

event.target

A handler on a parent element can always get the details about where it actually happened.

The most deeply nested element that caused the event is called a *target* element, accessible as `event.target`.

Note the differences from this (`=event.currentTarget`):

- `event.target` – is the “target” element that initiated the event, it doesn't change through the bubbling process.
- `this` – is the “current” element, the one that has a currently running handler on it.

For instance, if we have a single handler `form.onclick`, then it can “catch” all clicks inside the form. No matter where the click happened, it bubbles up to `<form>` and runs the handler.

In `form.onclick` handler:

- `this` (`=event.currentTarget`) is the `<form>` element, because the handler runs on it.
- `event.target` is the actual element inside the form that was clicked.

Check it out:

Result

script.js

example.css

index.html

It's possible that `event.target` could equal `this` – it happens when the click is made directly on the `<form>` element.

Stopping bubbling

A bubbling event goes from the target element straight up. Normally it goes upwards till `<html>`, and then to document object, and some events even reach window, calling all handlers on the path.

But any handler may decide that the event has been fully processed and stop the bubbling.

The method for it is `event.stopPropagation()`.

For instance, here `body.onclick` doesn't work if you click on `<button>`:

```
<body onclick="alert(` the bubbling doesn't reach here `)">
  <button onclick="event.stopPropagation()">Click me</button>
</body>
```

event.stopImmediatePropagation()

If an element has multiple event handlers on a single event, then even if one of them stops the bubbling, the other ones still execute.

In other words, `event.stopPropagation()` stops the move upwards, but on the current element all other handlers will run.

To stop the bubbling and prevent handlers on the current element from running, there's a method `event.stopImmediatePropagation()`. After it no other handlers execute.

Don't stop bubbling without a need!

Bubbling is convenient. Don't stop it without a real need: obvious and architecturally well thought out.

Sometimes `event.stopPropagation()` creates hidden pitfalls that later may become problems.

For instance:

1. We create a nested menu. Each submenu handles clicks on its elements and calls `stopPropagation` so that the outer menu won't trigger.
2. Later we decide to catch clicks on the whole window, to track users' behavior (where people click). Some analytic systems do that. Usually the code uses `document.addEventListener('click',...)` to catch all clicks.
3. Our analytic won't work over the area where clicks are stopped by `stopPropagation`. Sadly, we've got a "dead zone".

There's usually no real need to prevent the bubbling. A task that seemingly requires that may be solved by other means. One of them is to use custom events, we'll cover them later. Also we can write our data into the event object in one handler and read it in another one, so we can pass to handlers on parents information about the processing below.

Capturing

There's another phase of event processing called "capturing". It is rarely used in real code, but sometimes can be useful.

The standard [DOM Events](#) describes 3 phases of event propagation:

1. Capturing phase – the event goes down to the element.
2. Target phase – the event reached the target element.
3. Bubbling phase – the event bubbles up from the element.

Here's the picture, taken from the specification, of the capturing (1), target (2) and bubbling (3) phases for a click event on a `<td>` inside a table:

That is: for a click on `<td>` the event first goes through the ancestors chain down to the element (capturing phase), then it reaches the target and triggers there (target phase), and then it goes up (bubbling phase), calling handlers on its way.

Until now, we only talked about bubbling, because the capturing phase is rarely used.

In fact, the capturing phase was invisible for us, because handlers added using `on<event>`-property or using HTML attributes or using two-argument `addEventListener(event, handler)` don't know anything about capturing, they only run on the 2nd and 3rd phases.

To catch an event on the capturing phase, we need to set the handler capture option to true:

```
elem.addEventListener(..., {capture: true})
```

// or, just "true" is an alias to {capture: true}

```
elem.addEventListener(..., true)
```

There are two possible values of the capture option:

- If it's false (default), then the handler is set on the bubbling phase.
- If it's true, then the handler is set on the capturing phase.

Note that while formally there are 3 phases, the 2nd phase ("target phase": the event reached the element) is not handled separately: handlers on both capturing and bubbling phases trigger at that phase.

Let's see both capturing and bubbling in action:

```
<style>
```

```
body * {  
  margin: 10px;  
  border: 1px solid blue;  
}
```

```
</style>
```

```
<form>FORM
```

```
<div>DIV
```

```
<p>P</p>
```

```
</div>
```

```
</form>
```

```
<script>
```

```

for(let elem of document.querySelectorAll('*')) {
  elem.addEventListener("click", e => alert(` Capturing: ${elem.tagName}` ), true);
  elem.addEventListener("click", e => alert(` Bubbling: ${elem.tagName}` ));
}
</script>

```

The code sets click handlers on every element in the document to see which ones are working.

If you click on <p>, then the sequence is:

1. HTML → BODY → FORM → DIV → P (capturing phase, the first listener):
2. P → DIV → FORM → BODY → HTML (bubbling phase, the second listener).

Please note, the P shows up twice, because we've set two listeners: capturing and bubbling. The target triggers at the end of the first and at the beginning of the second phase.

There's a property `event.eventPhase` that tells us the number of the phase on which the event was caught. But it's rarely used, because we usually know it in the handler.

To remove the handler, `removeEventListener` needs the same phase

If we `addEventListener(..., true)`, then we should mention the same phase in `removeEventListener(..., true)` to correctly remove the handler.

Listeners on the same element and same phase run in their set order

If we have multiple event handlers on the same phase, assigned to the same element with `addEventListener`, they run in the same order as they are created:

```

elem.addEventListener("click", e => alert(1)); // guaranteed to trigger first
elem.addEventListener("click", e => alert(2));

```

The `event.stopPropagation()` during the capturing also prevents the bubbling

The `event.stopPropagation()` method and its sibling `event.stopImmediatePropagation()` can also be called on the capturing phase. Then not only the further capturing is stopped, but the bubbling as well.

In other words, normally the event goes first down ("capturing") and then up ("bubbling"). But if `event.stopPropagation()` is called during the capturing phase, then the event travel stops, no bubbling will occur.